Mariana Agudelo Salazar - A00398722
Natalia Vargas - A00398706

Taller 1: Construcción de pipelines en Cloud

Metodología y Estrategias de Control de Versiones

1. **Kanban:** Para abordar el ciclo de vida de este proyecto, que integra de manera simultánea el desarrollo de la aplicación y el despliegue de infraestructura, el equipo ha adoptado Kanban como metodología ágil. Esta decisión se fundamenta en la necesidad de mantener un flujo de trabajo continuo, flexible y altamente visual. Dado que las tareas de desarrollo (Dev) y operaciones (Ops) comparten dependencias directas, un tablero Kanban permite mapear el flujo de valor a través de estados claros (por ejemplo: Backlog, En Progreso - Dev, En Progreso - Ops, Pruebas, Listo para Despliegue y Completado). Esto facilita la identificación rápida de cuellos de botella, limita el trabajo en curso (WIP) y asegura la entrega continua y estable de valor.
2. **Estrategia de Branching para Desarrolladores**: Para la gestión del código fuente de los diferentes componentes (microservicios), se implementará la estrategia ***Git Flow***. Esta metodología proporciona un marco robusto y estructurado, lo cual es ideal para gestionar el ciclo de vida de múltiples servicios interconectados y garantizar la estabilidad del sistema. La convención de ramas se define de la siguiente manera:
    a. **main**: Rama principal que contiene exclusivamente código estable y en estado de producción.
    b. **develop**: Rama de integración donde se unifica y prueba todo el código nuevo antes de su paso a producción.
    c. **feature/***: Ramas efímeras creadas a partir de develop para aislar el desarrollo de nuevas características o tareas específicas. Al finalizar, se integran de nuevo a develop mediante un *Pull Request* (PR) que debe superar las pruebas automatizadas del pipeline.
    d. **hotfix/***: Ramas de emergencia creadas a partir de main para solucionar errores críticos en producción. Una vez resuelto el fallo, se integran tanto en main como en develop para mantener el historial sincronizado.
3. **Estrategia de Branching para Operaciones:** Para el manejo seguro de la Infraestructura como Código (IaC), configuraciones de contenedores y definición de pipelines, se adoptará una estrategia de ***Ramas por Entorno*** (Environment-based Branching). Esta separación lógica permite aislar los cambios arquitectónicos y probarlos de manera exhaustiva antes de impactar el sistema principal. La estructura se maneja así:
    a. **staging** (entorno de pre-producción): Refleja un entorno de pruebas idéntico a producción. Cualquier modificación en los recursos de la nube o en los pipelines se fusiona y despliega primero en esta rama para validar su correcta ejecución junto con la rama develop de la aplicación.
    b. **production** (entorno principal): Refleja la infraestructura definitiva. Los cambios de infraestructura solo se fusionan hacia esta rama tras haber sido validados exitosamente en staging, desencadenando el pipeline de infraestructura definitivo.

Diseño de la solución

1. **Patrones de Diseño de Nube**

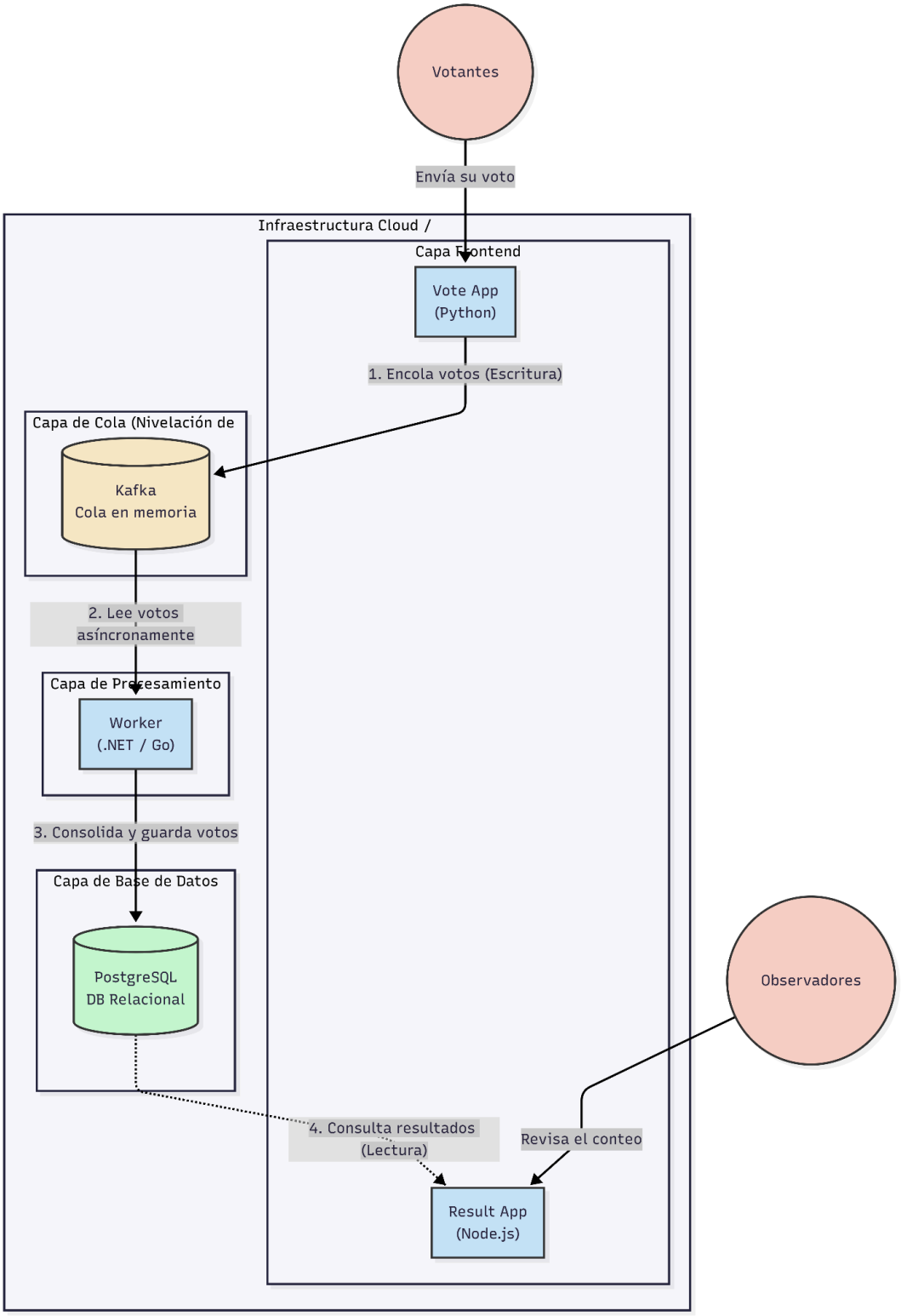
    Para este proyecto de votación distribuida, podemos justificar claramente la implementación de estos dos patrones de diseño de nube reconocidos en la industria:

- **Queue-Based Load Leveling** (Nivelación de carga basada en colas)
    - ***Justificación en el proyecto:*** En la aplicación, el servicio de votación (Vote App) recibe las peticiones de los usuarios y las envía a Kafka en lugar de escribirlas directamente en la base de datos
    - ***Valor:*** Kafka actúa como un amortiguador. Si hay un pico masivo de usuarios votando al mismo tiempo, el sistema no colapsa ni satura la base de datos; la cola en memoria absorbe la carga y el servicio Worker procesa y guarda los votos a su propio ritmo
    - ***Implementación:*** El servicio Vote (Java / Spring Boot) publica mensajes en el topic "votes" de Kafka. El Worker (Go) consume ese topic de forma asíncrona usando la librería Sarama y persiste cada voto en PostgreSQL
- **CQRS** (Command and Query Responsibility Segregation / Segregación de Responsabilidades de Comando y Consulta)
    - ***Justificación en el proyecto:*** La arquitectura separa claramente la acción de enviar datos (votar) de la acción de leer datos (ver resultados finales). Esta separación está implementada en el repositorio base y se justifica como patrón de diseño adoptado.
    - ***Valor:*** El servicio Vote se encarga exclusivamente de las operaciones de escritura (comandos) a través de la cola Kafka y el Worker, mientras que el servicio Result se encarga exclusivamente de las operaciones de lectura (consultas) conectándose directamente a la base de datos PostgreSQL. Esto permite escalar ambos servicios de forma independiente (por ejemplo, si mucha gente entra a ver los resultados pero ya nadie está votando).
    - ***Implementación:***
        - Vote App → Kafka → Worker → PostgreSQL (escritura / Commands).
        - Result App → PostgreSQL (lectura / Queries). Los dos flujos son completamente independientes

Mariana Agudelo Salazar - A00398722
Natalia Vargas - A00398706

2. Diagrama de Arquitectura

La siguiente figura muestra el diagrama de arquitectura de la solución, con las tecnologías reales utilizadas por cada servicio:



El flujo de datos sigue el siguiente recorrido:

- El usuario accede al servicio voting-app (Java / Spring Boot) en el puerto 8080 y emite su voto.
- El voto es publicado como mensaje al topic "votes" en Kafka (patrón Queue-Based Load Leveling). Kafka absorbe los picos de tráfico sin saturar la base de datos.
- El servicio worker (Go) consume el topic de Kafka de forma asíncrona y persiste cada voto en la base de datos.
- Los votos son almacenados en PostgreSQL en la tabla "votes".
- El servicio result-app (Node.js) consulta PostgreSQL cada segundo (patrón CQRS — solo lectura) y emite los resultados en tiempo real a los observadores vía Socket.IO en el puerto 4000